Project Blueprint: Secure Digital Evidence & Legal Document Management System (SIH26190)

1. Problem Definition & Operational Context

- **Problem Statement ID:** SIH26190
- **Nodal Ministry:** Ministry of Home Affairs (MHA)
- **Target Stakeholders:** Investigating Officers (IOs), Station House Officers (SHOs), Forensic Science Laboratories (FSL), Public Prosecutors, and Judicial Magistrates.

The Indian criminal justice lifecycle relies heavily on physical documentation (First Information Reports (FIRs), Case Diaries, Seizure Memos, Charge Sheets, and Forensic Chemical/Ballistic Reports). When digitized, these documents are commonly stored in fragmented departmental systems, shared across insecure email/messaging channels, or kept on generic local drives. This fragmented workflow introduces three systemic vulnerabilities:

- **Evidentiary Compromise:** Digital evidence submitted in court faces admissibility challenges under Section 63 of the Bharatiya Sakshya Adhiniyam, 2023 (BSA) if the prosecution cannot prove an unbroken, tamper-evident chain of custody from the point of collection.
- **Insider Threat & Retroactive Tampering:** Standard database administrators or unauthorized users can silently alter, replace, or backdate file records without generating an unalterable audit trail.
- **Information Leaks:** Uncontrolled downloads and lack of granular access controls lead to leaks of sensitive witness statements, identities of vulnerable victims, or ongoing investigation strategies.

2. Proposed Solution: Architectural Overview

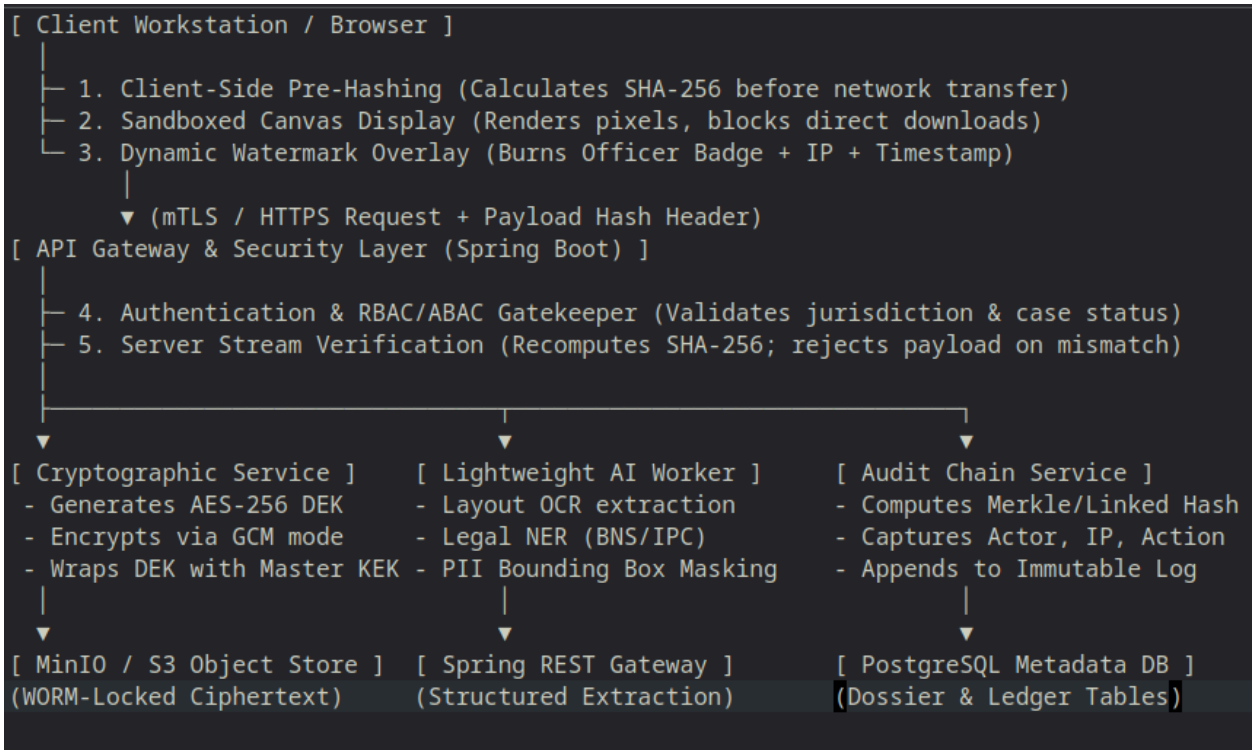
The proposed platform is an **Enterprise Secure Digital Evidence & Document Repository** designed around zero-trust access, client-to-storage cryptographic sealing, and immutable audit logs.

Rather than functioning as a standard file storage drive, the system treats every file as a **forensic artifact**:

- Every file is pre-hashed at the browser level before network transmission to prevent in-flight tampering.
- Every document is encrypted at rest using envelope encryption (unique single-use data key per file, locked under a master key).
- Every access, view, export, or status change appends a cryptographically linked block to an internal hash ledger, making unauthorized database edits immediately noticeable.
- The frontend streams content into an isolated, canvas-based sandbox that applies dynamic, viewer-specific forensic watermarks to prevent data exfiltration.
- A lightweight AI sidecar handles routine operational tasks: automated parsing of statutory legal sections (BNS/IPC), document categorization, and automated PII masking for public

defense disclosures.

3. System Architecture & High-Level Data Flow



Step-by-Step Data Flow

1. **Upload & Verification:** The officer selects an evidence document. The browser calculates the file's SHA-256 checksum locally and sends it in the HTTP headers alongside the file stream.
2. **Server-Side Integrity Check:** The Spring Boot backend consumes the incoming byte stream through an active digest stream, independently calculating the hash. If the client hash and server hash differ, the connection terminates immediately.
3. **Envelope Encryption & Storage:** The backend generates an ephemeral AES-256 symmetric data key, encrypts the file stream in Galois/Counter Mode (GCM), encrypts the data key using a master Key Encryption Key (KEK), and writes the encrypted payload into a Write-Once-Read-Many (WORM) MinIO storage bucket.
4. **AI Parsing (Async):** An asynchronous worker sends the file to the lightweight AI engine. The engine runs optical character recognition, extracts statutory sections, and flags sensitive details (Aadhaar numbers, phone numbers, minor names) for redaction.
5. **Chain-of-Custody Sealing:** The transaction writes to the PostgreSQL audit ledger. The new block includes the hash of the preceding ledger entry, binding the event into an unbroken, tamper-evident hash chain.
6. **Controlled Delivery:** When an authorized user accesses the document, the backend verifies their permissions against the active case status, streams the decrypted content

into the frontend canvas viewer, and logs a READ event in the audit chain.

4. Backend Architecture Plan (Spring Boot Ecosystem)

The backend handles security enforcement, business workflows, cryptographic pipelines, and transactional database integrity.

Security & Access Control (RBAC + ABAC)

- **Authentication:** Stateless JSON Web Tokens (JWT) containing the user's unique identifier, rank, assigned police station code, and district.
- **Role-Based Access Control (RBAC):** Strict operational roles (Investigating Officer, Station House Officer, Forensic Examiner, Public Prosecutor, Judicial Magistrate).
- **Attribute-Based Access Control (ABAC):** Dynamic security evaluations:
    - *Jurisdiction Boundary:* An officer can only access dockets registered within their designated police station or district unless explicit transfer orders exist.
    - *Case Lifecycle State:* When a docket's status changes from UNDER_INVESTIGATION to CHARGESHEET_FILED, the Investigating Officer's permissions drop to read-only, preventing retroactive edits to the case diary.
    - *Document Confidentiality Level:* Unredacted witness statements and autopsy photographs require elevated authorization flags.

Cryptographic & Storage Subsystem

- **Envelope Encryption Architecture:**
    - Master Key Encryption Key (KEK) is stored securely in environment variables or an enterprise key management system (KMS).
    - For every uploaded file, a single-use 256-bit Data Encryption Key (DEK) and a 12-byte initialization vector (IV) are generated via secure random generators.
    - The file stream is encrypted using AES/GCM/NoPadding.
    - The encrypted file is saved to storage, and the encrypted DEK (wrapped using the KEK) is stored alongside the document metadata in PostgreSQL.
- **WORM Object Storage Strategy:**
    - Uses MinIO (S3-compatible) configured with Object Lock in Compliance Mode. This prevents anyone—including the storage admin—from deleting or overwriting the file for a predetermined retention period (e.g., 20 years).

Append-Only Audit Ledger & Chain of Custody

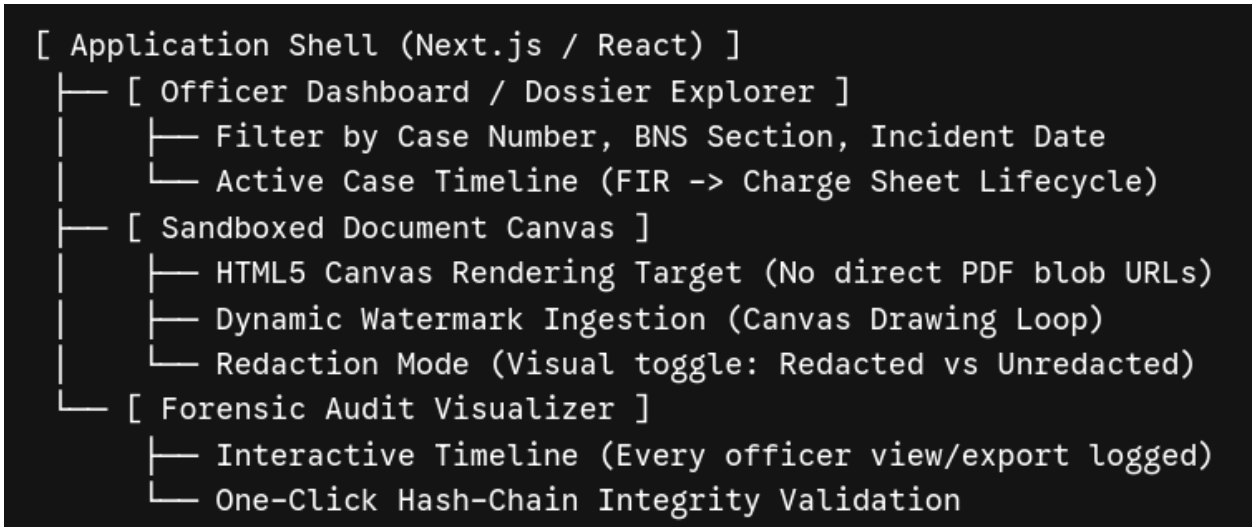
- **Hash-Chained Audit Engine:** Every access, annotation, or status change creates a new record containing:
  {Current Hash} ={SHA256}({Previous Hash} +{Document ID} + {Actor ID} + {Action Type} + {Timestamp})
- **Tamper Detection Mechanism:** The system provides an automated verification endpoint that recalculates the hash chain sequentially from the genesis block. If any database entry has been manually edited or deleted, the computed hash mismatches the stored hash, immediately flagging the record as compromised.

Statutory Evidence Certification (BSA Section 63)

- An automated reporting service built using OpenPDF generates an official Electronic Evidence Certificate for court submissions.
- The certificate includes the document's permanent forensic hash, system ingestion timestamp, certifying officer's credentials, server node identifiers, and a verifiable QR code linking directly to the system's audit verification view.

5. Frontend Architecture Plan (React / Next.js)

The frontend provides an intuitive, high-efficiency interface for law enforcement and legal teams while preventing client-side data leaks.



Core Frontend Modules

- **Sandboxed Document Canvas:**
  - Renders PDF pages or evidence imagery directly onto an HTML5 Canvas using PDF.js.
  - Avoids embedding native browser PDF viewers, preventing users from clicking default browser download or print controls.
  - Disables standard right-click context menus, image drags, and text selection on sensitive pages.
- **Dynamic Forensic Watermarking:**
  - Draws an indelible, semi-transparent diagonal watermark across the canvas rendering pipeline.
  - Displays the viewing officer's badge number, official station code, user IP address, and access timestamp.
  - If a user photographs their screen with a smartphone, the resulting image clearly identifies who accessed the record.
- **Interactive Chain-of-Custody Visualizer:**
  - A clear, chronological timeline showing the entire history of an evidence file: when it

was created, who inspected it, when it went to the lab, and when it was submitted to court.
    - Displays green verification indicators for valid cryptographic signatures and red warnings if any tampering is detected.
- **Dual-Pane Officer Workspace:**
    - Left pane: High-resolution canvas viewer for inspecting documents.
    - Right pane: Metadata dashboard showing extracted legal sections, accused details, and an audit trail sidebar.

6. AI/ML Subsystem Plan (Lightweight Python Microservice)

The AI component is designed as an isolated micro-worker that processes requests asynchronously, preventing heavy computer vision dependencies from impacting the core Spring Boot backend.

Responsibilities & Boundaries

- **OCR Ingestion Pipeline:** Uses PaddleOCR to extract text coordinates, words, and structural layouts from scanned FIRs, typewritten reports, and handwritten notes.
- **Statutory Named Entity Recognition (NER):**
    - Identifies legal parameters using custom regex patterns and lightweight NLP models.
    - Extracts statutory references under the Bharatiya Nyaya Sanhita (BNS) / Indian Penal Code (IPC), court docket numbers, and police station codes.
- **Automated PII Redaction Engine:**
    - Detects Personally Identifiable Information (PII) such as 12-digit Aadhaar numbers, phone numbers, bank accounts, and victim identities in sensitive cases.
    - Generates pixel-accurate 2D bounding boxes [min_x, min_y, max_x, max_y] for flagged items.
    - The backend uses these coordinates to generate clean, redacted copies for defense counsel and public disclosures without altering the original, sealed evidence file.

7. Core Data Models (Conceptual Reference)

Entity Name	Key Attributes	Functional Role
CaseDocket	id, caseNumber, stationCode, currentStatus, primaryIoId, createdAt, lockedAt	Central container for a single criminal investigation/case file.
CaseDocument	id, docketId, docType,	Evidentiary files associated

	s3Uri, fileSizeBytes, clientHash, serverHash, encryptedDek, iv, uploadedBy, isSealed	with a docket (FIR, Case Diary, Autopsy Report).
AuditLedgerChain	id, documentId, sequenceNumber, actorUserId, actionType, timestamp, previousBlockHash, currentBlockHash	Immutable, cryptographically chained record of all document events.
AccessPermission	id, docketId, userId, roleScope, grantedBy, validUntil	Dynamic permissions granting temporary access to specific dockets.

8. The Live Pitch & Presentation Strategy

1. **The Ingestion Pitch:** Show an Investigating Officer uploading an FIR scan. Highlight the client-side hash generation, the envelope encryption pipeline, and the instant extraction of statutory BNS sections by the AI sidecar.
2. **The Security Pitch:** Switch to a defense lawyer's account to demonstrate how the platform displays an automatically redacted copy that protects victim identities while keeping the original file intact.
3. **The Live Tamper Demonstration:**
    - Open the database management tool or terminal live on screen.
    - Manually edit a single byte or character in the database or storage layer to simulate an insider attack.
    - Refresh the officer’s dashboard and run the integrity check.
    - Show the system flagging the record in red: **"INTEGRITY VIOLATION DETECTED: CHAIN OF CUSTODY BROKEN AT BLOCK #4"**.
4. **The Legal Close:** Click "Export Court Certificate" to generate the official Section 63 BSA compliance document with its verification QR code, proving the platform is ready for real courtroom use.

9. Curated Standards & Documentation References

- **Cryptographic Specifications & Guidelines:**
    - NIST Special Publication 800-38D (AES-GCM Specification): Standard for Galois/Counter Mode authenticated encryption and data confidentiality.
    - RFC 6962: Certificate Transparency / Merkle Tree Structures: Architectural reference for append-only, tamper-evident hash ledgers.
- **Backend Frameworks & Enterprise Standards:**

    - Spring Security Architecture Documentation: Reference for implementing method-level access controls and custom permission evaluators.
    - MinIO Object Storage Documentation: Guide for configuring bucket-level WORM policies and object-locking APIs.
- **AI & Document Processing Libraries:**
    - PaddleOCR Project Repository: Documentation for layout-aware optical character recognition and bounding box extraction.
    - OpenPDF GitHub Repository: LGPL/MPL-compliant Java library for programmatic PDF and legal certificate generation.
- **Statutory Criminal Law Reforms:**
    - Bharatiya Sakshya Adhiniyam, 2023 (BSA): Primary statute governing the admissibility of electronic records and Section 63 certificate requirements.
    - Bharatiya Nagarik Suraksha Sanhita, 2023 (BNSS): Procedural framework mandating digital case diaries, electronic summons, and timeline management for investigation records.